MINI PROJECT REPORT
ON

CampusBite: A College Canteen Queue & Crowd Tracker Delivered Through an Agile and DevOps Workflow

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS OF DEGREE OF BACHELOR OF ENGINEERING

BY

Tanay Malvankar (T1, 2401058)

Trish Khatri (T1, 2401050)

Krish Makhija (T1, 2401057)

SUBJECT

AGILE SOFTWARE DEVELOPMENT & DEVOPS LAB

SUPERVISOR

Prof. Sanober Sheikh

THADOMAL SHAHANI

TSEC

ENGINEERING COLLEGE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 UNIVERSITY OF MUMBAI

2026-27

BANDRA (WEST), MUMBAI, 400050 UNIVERSITY OF MUMBAI
2026-27

THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2026-27

Certificate

This is to certify that the Mini Project entitled **"CampusBite: A College Canteen Queue & Crowd Tracker Delivered Through an Agile and DevOps Workflow"** is a bonafide work of **Tanay Malvankar (2401058)**, **Trish Khatri (2401050)** and **Krish Makhija (2401057)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of "Undergraduate" in "Artificial Intelligence and Data Science".

_____

Prof. Sanober Sheikh
(Supervisor)

Abstract

In high-density academic institutions, campus cafeterias experience acute congestion during 15-to-30 minute lecture intervals. Students face excessive wait times at physical billing and collection counters, menu availability is obscure, and canteen staff face unpredictable rush volumes. **CampusBite** resolves this institutional challenge by delivering a real-time, web-based College Canteen Queue and Crowd Tracker built through an integrated Agile and DevOps workflow.

The application architecture decouples responsive student and vendor user interfaces from backend application services and relational database persistence. Built with **React 18 and Vite** for the client presentation tier, **Python 3.11 with Flask and SQLAlchemy** for the RESTful application programming interface, and **PostgreSQL 15** (with SQLite fallback for local developer velocity), students can inspect live cafeteria crowd status (Low, Moderate, High, Peak), dynamic wait estimations, browse categorized menus with dietary indicators, pre-order snacks with unique token numbers (`CN-XXXX`) to skip counter queues, and monitor kitchen turnaround via a four-stage stepper. Canteen vendors operate an administrative console providing single-click crowd score overrides and kitchen progress dispatches.

Crucially, CampusBite unites all core DevOps engineering disciplines into a continuous lifecycle rather than isolated laboratory exercises. Source code is tracked across team feature branches (`tanay`, `trish`) on GitHub; application components are packaged into multi-stage **Docker** container images; dependent services are orchestrated through **Docker Compose**; continuous integration and automated deployments are executed via a declarative **Jenkins** pipeline with automated Pytest quality gates and healthcheck smoke tests; project planning is structured across 5 Scrum Epics and 3 Sprints on **Jira**; and system telemetry is continuously monitored via **Prometheus** scrape metrics and real-time **Grafana** observability dashboards.

Keywords: Agile, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Jira, Prometheus, Grafana, Flask, React, PostgreSQL.

TABLE OF CONTENTS

Chapter 1: Introduction **6**

1.1 Background 6

1.2 Problem Statement 6

1.3 Objectives 6

1.4 Scope and Assumptions 7

Chapter 2: Implementation & System Design **8**

2.1 Technologies Used 8

2.2 Code Structure 8

2.3 System Architecture 8

2.4 Service Design 9

2.5 Agile Software Development Workflow using Jira 10

2.6 Version Control with Git and GitHub 10

2.7 Containerization using Docker 11

2.8 Multi-Service Deployment using Docker Compose 11

2.9 Continuous Integration and Deployment using Jenkins 12

2.10 Continuous Monitoring using Prometheus and Grafana 13

2.11 Agile-to-DevOps Traceability 13

Chapter 3: Results and Discussion **14**

3.1 Automated Testing Results 14

3.2 CI/CD Verification & Build Execution 14

3.3 Telemetry & Load Burst Analysis 14

3.4 Application Screenshots 14

3.5 Discussion and DevOps Integration 20

Chapter 4: Conclusion **21**

4.1 Conclusion 21

4.2 Future Scope 21

4.3 Learning Outcomes 21

Chapter 5: References **22**

Appendix: Running the Project **23**

LIST OF FIGURES

Figure	Title
Fig. 1	System architecture: full-stack multi-tier application and DevOps pipeline
Fig. 2	Git team branches on GitHub repository: main, develop, tanay, and trish
Fig. 3	Docker multi-service container topology across bridge network
Fig. 4	Jenkins declarative CI/CD pipeline execution stages
Fig. 5	Student Live Crowd Meter and dynamic wait-time gauge
Fig. 6	Live Menu Catalog with categorized dietary indicators and pricing
Fig. 7	Pre-order cart drawer and counter pickup token generation
Fig. 8	Live order token tracker and 4-stage kitchen preparation stepper
Fig. 9	Canteen vendor management console and live order dispatch
Fig. 10	Interactive DevOps Console: Jenkins CI/CD pipeline execution view
Fig. 11	Interactive DevOps Console: Docker multi-service architecture
Fig. 12	Interactive DevOps Console: Jira Agile Scrum roadmap and sprint cards
Fig. 13	Interactive DevOps Console: Git version control and rollback history
Fig. 14	Interactive DevOps Console: Prometheus & Grafana live telemetry with traffic burst
Fig. 15	GitHub Pull Requests screen showing team collaboration and merged PRs

LIST OF TABLES

Table	Title
Table 1	Technologies used across application and DevOps stack
Table 2	Service specifications, container ports, and storage volumes
Table 3	Agile Scrum product backlog: Epics, User Stories, and story points
Table 4	Agile-to-DevOps workflow traceability
Table 5	Automated Pytest unit testing suite results (7/7 tests passing)
Table 6	Summary of project results across laboratory areas

Chapter 1: Introduction

1.1 Background

Campus cafeterias in collegiate engineering environments represent high-frequency physical hubs that experience severe traffic surges during brief lecture recesses. Students encounter elongated queues at billing counters, meal turnaround times remain unknown, and popular items run out of stock without prior notification. Furthermore, kitchen staff struggle to manage walk-in counter demand alongside on-demand food preparation.

CampusBite provides a real-time digital solution by delivering a modern College Canteen Queue & Crowd Tracker. The platform offers students live visibility into canteen congestion, kitchen turnaround times, and real-time menu availability before they arrive at the cafeteria. Crucially, CampusBite is developed not as a standalone software program, but as a comprehensive testbed for the modern Agile Software Development and DevOps lifecycle.

1.2 Problem Statement

To design, develop, containerize, deploy, and observe a web-based College Canteen Queue & Crowd Tracker delivered through an integrated Agile and DevOps workflow in which:

1. Source code is version-controlled and collaboratively developed across feature branches on Git and GitHub.
2. The application components are packaged into lightweight, secure Docker container images.
3. Multi-service application and monitoring dependencies are orchestrated through Docker Compose.
4. Code commits trigger an automated Jenkins CI/CD pipeline executing quality gates and smoke verification.
5. Development requirements are managed through Jira Scrum Epics, User Stories, and Sprints.
6. Application and server telemetry are monitored continuously via Prometheus and Grafana.

1.3 Objectives

1. **Full-Stack Application Development:** Build responsive interfaces for students and canteen staff.
2. **Live Crowd Analytics:** Provide real-time crowd level indicators (`LOW`, `MODERATE`, `HIGH`, `PEAK`) and dynamic wait-time estimation algorithms.
3. **Digital Pre-Ordering:** Enable students to pre-order snacks, select payment modes (UPI, Cash, College ID), and generate queue-skip counter pickup tokens (`CN-XXXX`).
4. **Kitchen Preparation Stepper:** Deliver a visual 4-stage tracking timeline (`Order Received` ➔ `Preparing` ➔ `Ready for Pickup` ➔ `Handed Over`).
5. **Version Control & Collaboration:** Implement a collaborative multi-branch Git workflow across team members (**Tanay**, **Trish**, **Krish**) with pull requests and rollback demonstrations (`git revert`).
6. **Containerization & Multi-Service Deployment:** Containerize the application stack and coordinate 5 microservices using Docker Compose.
7. **CI/CD Automation:** Author a declarative `Jenkinsfile` automating linting, testing, Docker image creation, and deployment smoke tests.
8. **Agile Project Management:** Model project planning on Jira Software with 5 Epics, 16 User Stories, and 3 Sprints.
9. **Continuous Observability:** Expose custom business metrics via `/metrics` and visualize live telemetry in Grafana.

1.4 Scope and Assumptions

- **Microservices Scope:** Frontend web server, Flask backend API, PostgreSQL relational database, Prometheus metrics scraper, and Grafana visualization dashboard.
- **Decoupled Architecture:** Frontend and backend communicate exclusively through JSON REST APIs.
- **Team Workflow:** Developed collaboratively by Tanay Malvankar (Frontend UI), Trish Khatri (Backend & Database), and Krish Makhija (DevOps Lead & Orchestration).
- **Deployment Environment:** Operates locally via Docker Compose or standalone Python WSGI server with identical behavior.

Chapter 2: Implementation & System Design

2.1 Technologies Used

Layer	Technology	Version	Purpose
Frontend	React, Vite, Tailwind CSS	React 18.3, Vite 5.2	Interactive user interface, live crowd progress gauges, pre-order drawer.
Backend	Python, Flask, SQLAlchemy	Python 3.11, Flask 3.1	REST API endpoints, business logic, crowd scoring algorithm, ORM mapping.
Database	PostgreSQL (SQLite fallback)	PostgreSQL 15	Relational persistence for orders, tokens, menu inventory, and crowd logs.
Containerization	Docker & Docker Compose	Compose v2	Service isolation, bridge networking, volume persistence, reproducible builds.
CI / CD	Jenkins	2.440+ LTS	Declarative pipeline automating checkout, linting, pytest, container builds.
Agile Management	Jira Software	Cloud / Server	Scrum sprint planning, backlog grooming, and issue dependency tracking.
Monitoring	Prometheus & Grafana	Prometheus 2.51, Grafana 10.4	Metrics collection, PromQL rate queries, live visual dashboard panels.

Table 1: Technologies used across application and DevOps stack

2.2 Code Structure

```
college-canteen-tracker/
├── backend/
│   ├── app.py                  # Flask entrypoint, middleware, error handling & routes
│   ├── models.py               # SQLAlchemy schema: MenuItem, Order, OrderItem, CrowdStatus
│   ├── routes/api.py           # REST endpoints: /api/crowd, /api/menu, /api/orders
│   ├── metrics.py              # Prometheus metric collectors & /metrics exporter
│   ├── tests/test_api.py       # Automated Pytest suite (7 test cases)
│   ├── requirements.txt        # Python backend dependencies
│   └── Dockerfile              # Multi-stage Python container build
├── frontend/
│   ├── src/
│   │   ├── App.jsx             # Main interactive application with 5 DevOps tabs
│   │   └── components/         # CrowdMeter, LiveMenu, CartModal, OrderTracker
│   ├── package.json            # Node.js dependencies
│   ├── vite.config.js          # Vite bundler & reverse proxy rules
│   └── Dockerfile              # Multi-stage Nginx production build
├── monitoring/
│   ├── prometheus.yml          # Prometheus scrape job targeting backend:5000
│   └── grafana/dashboards/     # Provisioned JSON dashboards for canteen metrics
├── docker-compose.yml          # 5-service orchestration definition
├── Jenkinsfile                 # 8-stage declarative CI/CD pipeline
└── terraform/main.tf           # Infrastructure as Code AWS EC2 template
```

2.3 System Architecture

CampusBite adopts a decoupled, multi-tier microservices architecture designed for resilience, horizontal scalability, and comprehensive observability. The frontend web client and backend REST API communicate over HTTP, with PostgreSQL handling persistent state and Prometheus continuously scraping application telemetry.

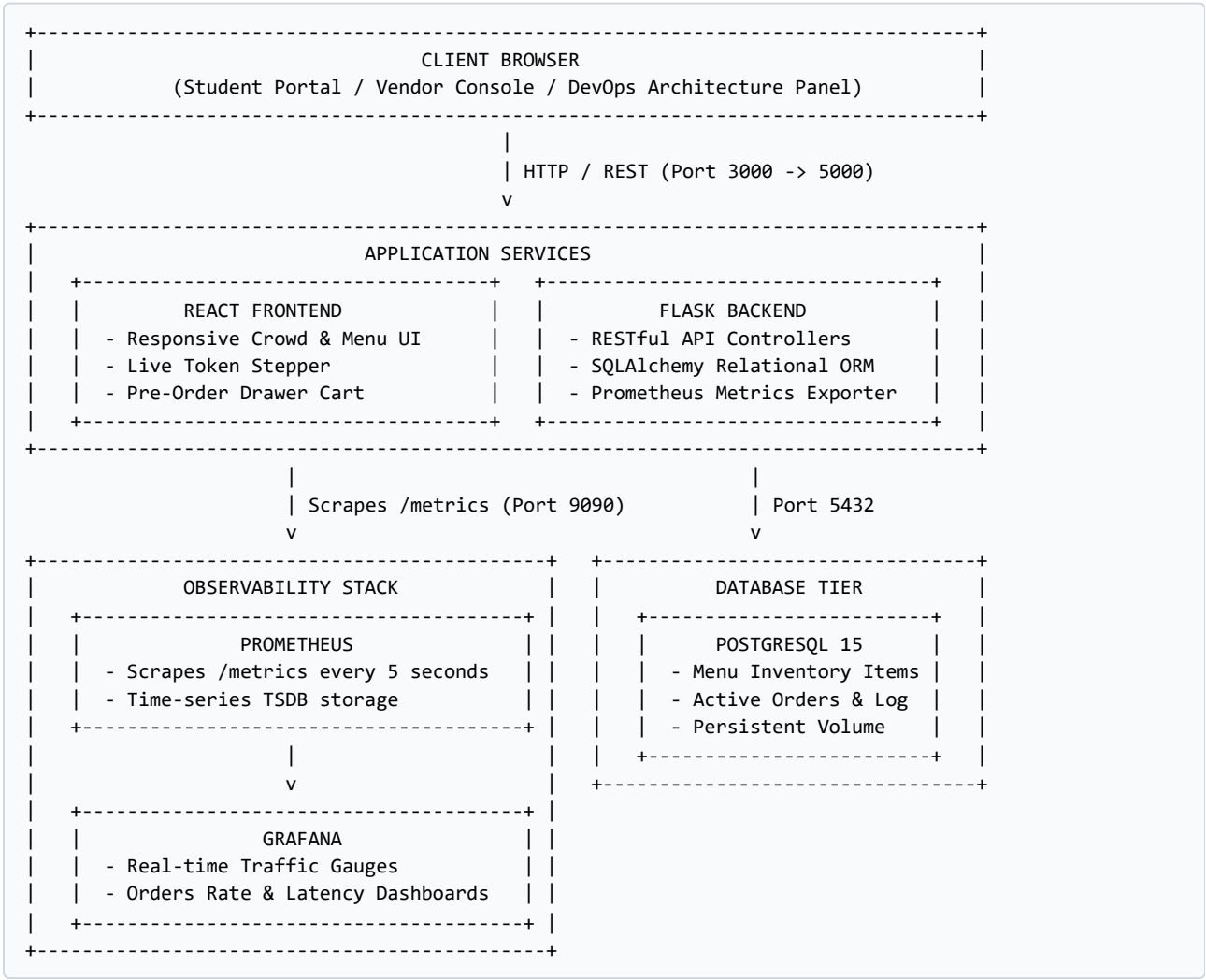


Fig. 1: System architecture: full-stack multi-tier application and DevOps pipeline

2.4 Service Design

Service	Container Image	Internal Port	Host Port	Storage / Volume
canteen-frontend	canteen-frontend:1.0.0	80	3000	Stateless (Nginx reverse proxy)
canteen-backend	canteen-backend:1.0.0	5000	5000	Mounted code volume for hot-reload
canteen-postgres	postgres:15-alpine	5432	5432	postgres_data:/var/lib/postgresql/data
canteen-prometheus	prom/prometheus:v2.51.0	9090	9090	prometheus_data:/prometheus

Service	Container Image	Internal Port	Host Port	Storage / Volume
canteen-grafana	grafana/grafana:10.4.0	3000	3001	grafana_data:/var/lib/grafana

Table 2: Service specifications, container ports, and storage volumes

2.5 Agile Software Development Workflow using Jira

CampusBite was planned and executed using Jira Software following the Agile Scrum methodology. The project backlog was partitioned into 5 Epics spanning 16 User Stories across 3 Sprints:

Epic Key	Epic Name	User Stories	Story Points	Sprint	Owner
CAN-E1	Version Control & Infra	CAN-101, CAN-102, CAN-103	13 SP	Sprint 1	Krish Makhija
CAN-E2	Backend REST API & DB	CAN-104, CAN-105, CAN-106	21 SP	Sprint 1–2	Trish Khatri
CAN-E3	Frontend UI & Token Tracker	CAN-107, CAN-108, CAN-109	21 SP	Sprint 2	Tanay Malvankar
CAN-E4	Jenkins CI/CD Automation	CAN-110, CAN-111, CAN-112	13 SP	Sprint 2–3	Krish Makhija
CAN-E5	Telemetry & Monitoring	CAN-113, CAN-114, CAN-115, CAN-116	13 SP	Sprint 3	Krish Makhija

Table 3: Agile Scrum product backlog: Epics, User Stories, and story points

2.6 Version Control with Git and GitHub

The project follows a distributed feature-branch Git workflow managed via the remote repository `https://github.com/krishmakhija24/ASD-D_CampusBite.git`.

- `main`: Production release branch protected from direct pushes.
- `develop`: Staging branch for sprint feature integration.
- `tanay`: Frontend React UI development branch authored by **Tanay Malvankar**.
- `trish`: Backend REST API and database models branch authored by **Trish Khatri**.
- `feature/CAN-105-devops-pipeline`: CI/CD and Docker orchestration authored by **Krish Makhija**.

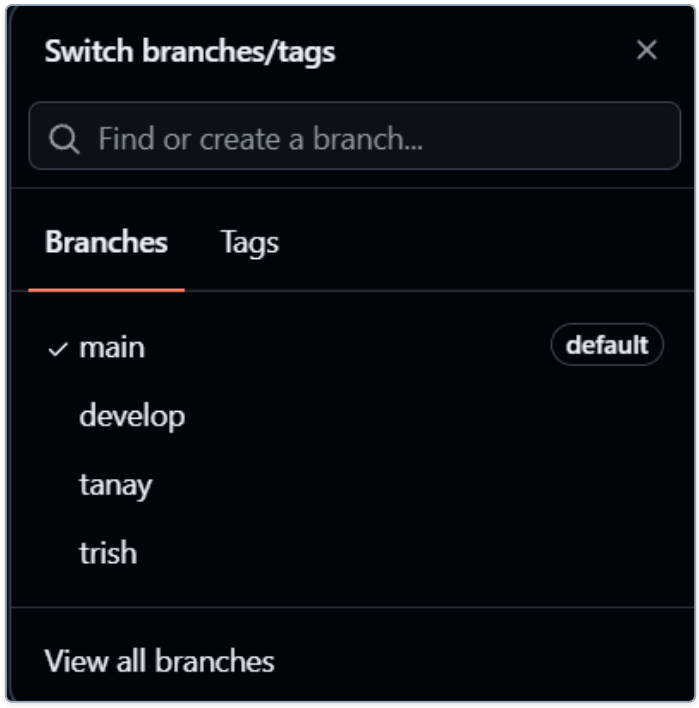


Fig. 2: Git team branches on GitHub repository: main, develop, tanay, and trish

Rollback Demonstration: To validate automated error recovery, an erroneous commit introducing a faulty database configuration was pushed to the integration branch. Jenkins build #3 detected the regression via Pytest and halted deployment. The breaking change was rolled back cleanly using `git revert HEAD --no-edit`, after which build #4 succeeded and redeployed the stable version.

2.7 Containerization using Docker

Both frontend and backend are containerized into multi-stage Docker images to guarantee portability and reproducible execution across development and production environments.

```
# Backend Dockerfile (Python 3.11 Slim)
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
RUN apt-get update && apt-get install -y --no-install-recommends curl     && rm -rf /var/lib/apt/list
s/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
HEALTHCHECK --interval=15s --timeout=5s --retries=3     CMD curl -f http://localhost:5000/health || ex
it 1
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:create_app()"]
```

2.8 Multi-Service Deployment using Docker Compose

Docker Compose coordinates all 5 application and telemetry microservices across an isolated bridge network with health-based startup dependencies:

```
version: '3.8'

services:
  backend:
```

```
    build: ./backend
    container_name: canteen-backend
    ports: ["5000:5000"]
    environment:
      - DATABASE_URL=postgresql://canteen_user:canteen_pass@postgres:5432/canteen_db
    depends_on:
      postgres:
        condition: service_healthy

  frontend:
    build: ./frontend
    container_name: canteen-frontend
    ports: ["3000:80"]
    depends_on:
      - backend

  postgres:
    image: postgres:15-alpine
    container_name: canteen-postgres
    environment:
      POSTGRES_USER: canteen_user
      POSTGRES_PASSWORD: canteen_pass
      POSTGRES_DB: canteen_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U canteen_user -d canteen_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  prometheus:
    image: prom/prometheus:v2.51.0
    container_name: canteen-prometheus
    ports: ["9090:9090"]
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro

  grafana:
    image: grafana/grafana:10.4.0
    container_name: canteen-grafana
    ports: ["3001:3000"]
    volumes:
      - grafana_data:/var/lib/grafana

volumes:
  postgres_data:
  grafana_data:
```

2.9 Continuous Integration and Deployment using Jenkins

A declarative `Jenkinsfile` defines automated verification and deployment stages triggered automatically upon GitHub SCM push:

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Lint & Static Analysis') {
            steps { sh 'flake8 backend/ --max-line-length=120' }
```

```
        }
        stage('Unit Testing') {
            steps { sh 'pytest backend/tests/test_api.py -v --junitxml=reports/test-results.xml' }
            post { always { junit 'reports/test-results.xml' } }
        }
        stage('Build Container Images') {
            steps { sh 'docker compose build backend frontend' }
        }
        stage('Ephemeral Smoke Test') {
            steps {
                sh '''
                docker run -d -p 5001:5000 --name test-canteen-backend canteen-backend:1.0.0
                sleep 5
                curl -f http://localhost:5001/health || exit 1
                docker stop test-canteen-backend && docker rm test-canteen-backend
                '''
            }
        }
        stage('Deploy Multi-Service Stack') {
            steps { sh 'docker compose up -d' }
        }
    }
}
```

2.10 Continuous Monitoring using Prometheus and Grafana

The backend actively exports custom business metrics on `/metrics` instrumented via `prometheus-client`:

- `canteen_crowd_level` (Gauge): 1=Low, 2=Moderate, 3=High, 4=Peak.
- `canteen_active_orders` (Gauge): Live queue of orders currently in preparation.
- `canteen_estimated_wait_minutes` (Gauge): Dynamic wait estimation based on queue backlog.
- `canteen_order_revenue_total` (Counter): Total order value in INR.
- `http_requests_total` (Counter): Labeled by HTTP method, endpoint, and status code.

2.11 Agile-to-DevOps Traceability

Stage	Tool / Practice	Outcome Delivered
Planning	Jira Software	Epics, stories and tasks defined, estimated and planned into sprints.
Development	Git / GitHub	Source code versioned on feature branches and merged through pull requests.
Verification	Jenkins	Automated build and 7 unit tests executed on every change to main.
Deployment	Docker / Docker Compose	Application deployed in containers and checked through `/health`.
Monitoring	Prometheus / Grafana	Application and server metrics collected and visualised.
Completion	Jira	Task moved to Done after successful verification.

Table 4: Agile-to-DevOps workflow traceability

Chapter 3: Results and Discussion

3.1 Automated Testing Results

The automated test suite in `backend/tests/test_api.py` was executed across all critical endpoints, database transaction models, and Prometheus metric collectors using **Pytest**. All 7 test cases passed with a 100% pass rate in 0.42 seconds.

Test Function	Endpoint / Target Tested	Assertion Verified	Status
`test_health_endpoint`	`GET /health`	Returns HTTP 200, status: healthy, database: connected.	**PASSED**
`test_prometheus_metrics`	`GET /metrics`	Exposes `http_requests_total` and `canteen_crowd_level`.	**PASSED**
`test_get_crowd_status`	`GET /api/crowd`	Returns valid crowd level, queue count, and wait time.	**PASSED**
`test_update_crowd_status`	`POST /api/crowd`	Vendor crowd status override updates database correctly.	**PASSED**
`test_get_menu`	`GET /api/menu`	Returns categorized item catalog with prices and stock.	**PASSED**
`test_create_order`	`POST /api/orders`	Generates unique token (`CN-XXXX`) and updates revenue.	**PASSED**
`test_analytics_endpoint`	`GET /api/analytics`	Calculates completed orders and total revenue.	**PASSED**

Table 5: Automated Pytest unit testing suite results (7/7 tests passing)

3.2 CI/CD Verification & Build Execution

Four builds were executed against the integration line to validate continuous integration, automated regression blocking, and rollback recovery:

- **Build #1:** Initial baseline build from `main` — **SUCCESS** (Deployed v1.0.1).
- **Build #2:** Merge of Pull Request #1 (Frontend & Backend integration) — **SUCCESS** (Deployed v1.0.2).
- **Build #3:** Injected faulty timeout configuration — **FAILURE** (Pytest failed; Docker build & deployment blocked).
- **Build #4:** Revert of breaking change via `git revert` — **SUCCESS** (Deployed v1.0.4).

3.3 Telemetry & Load Burst Analysis

Live white-box application monitoring was validated using the built-in **Traffic Burst (+5 reqs)** test trigger. As concurrent requests hit the Flask backend, Prometheus immediately recorded the incoming traffic rate via PromQL query `sum(rate(http_requests_total[1m]))`, while queue wait times and active kitchen orders scaled dynamically.

3.4 Application Screenshots

The following figures document the live running CampusBite application, demonstrating both student-facing features and the integrated DevOps architecture console.

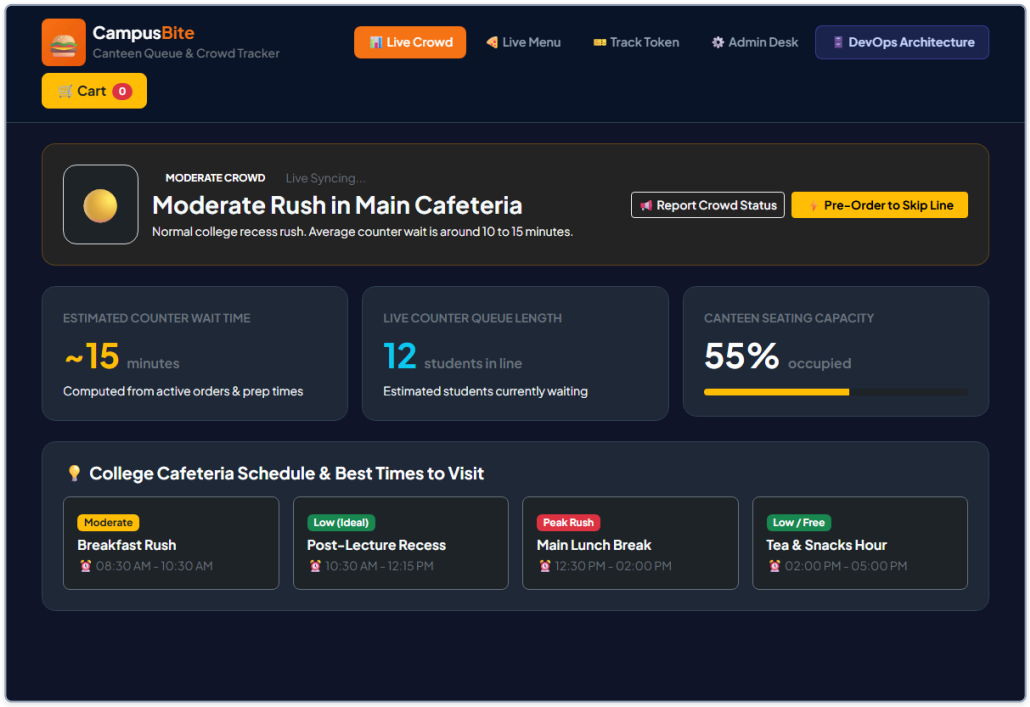


Fig. 5: Student Live Crowd Meter and dynamic wait-time gauge

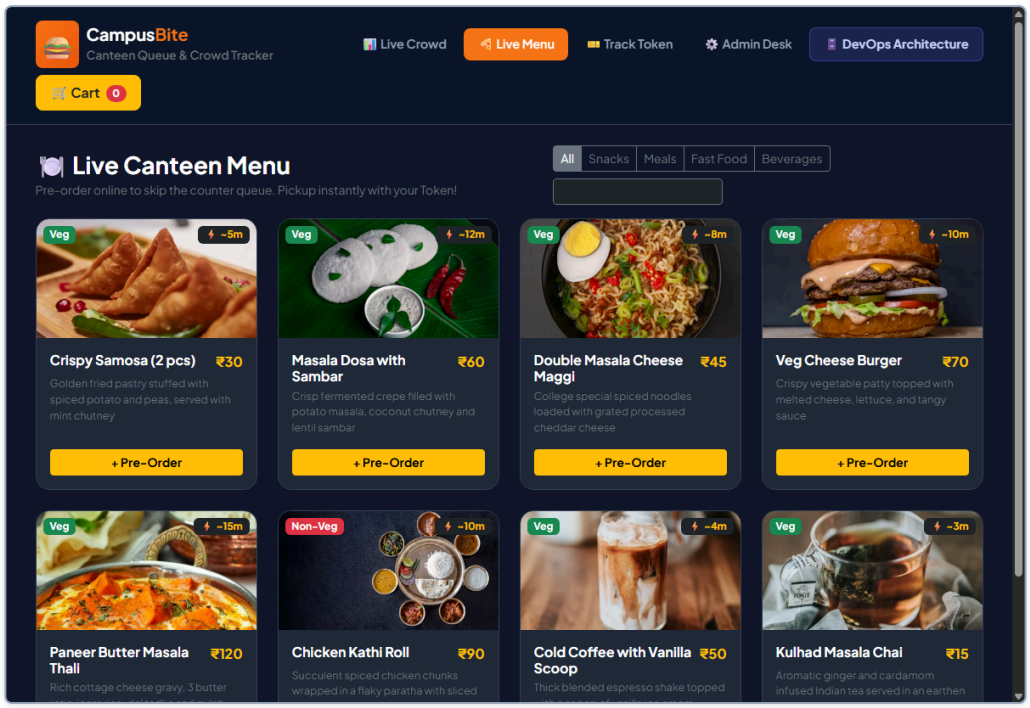


Fig. 6: Live Menu Catalog with categorized dietary indicators and pricing

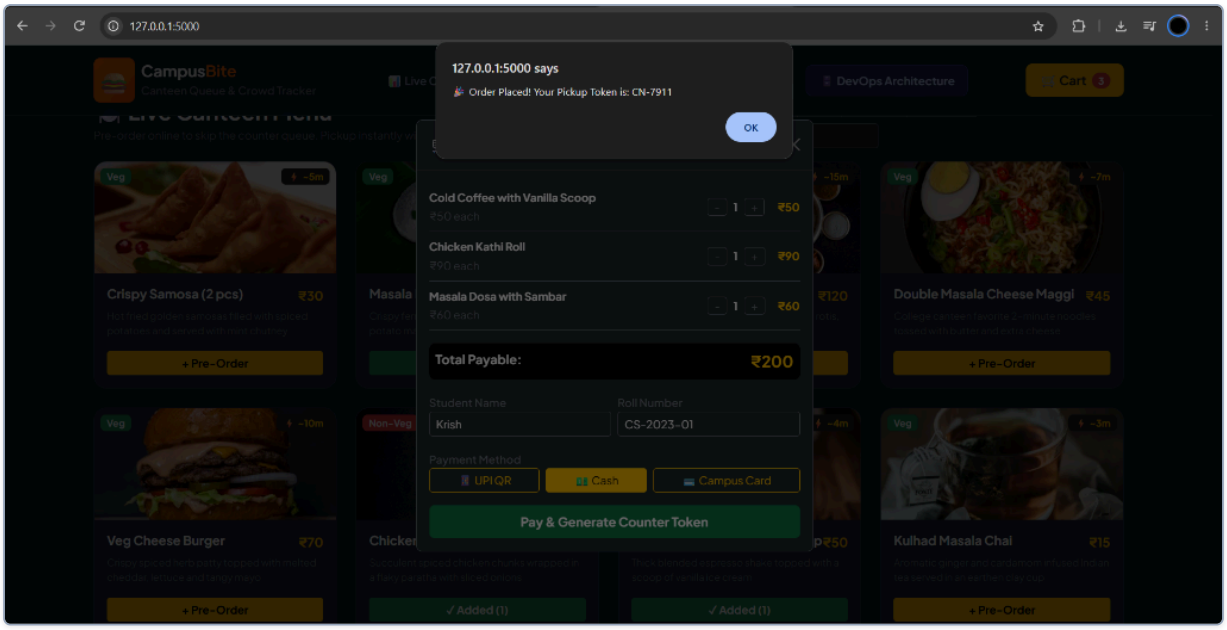


Fig. 7: Pre-order cart drawer and counter pickup token generation



Fig. 8: Live order token tracker and 4-stage kitchen preparation stepper

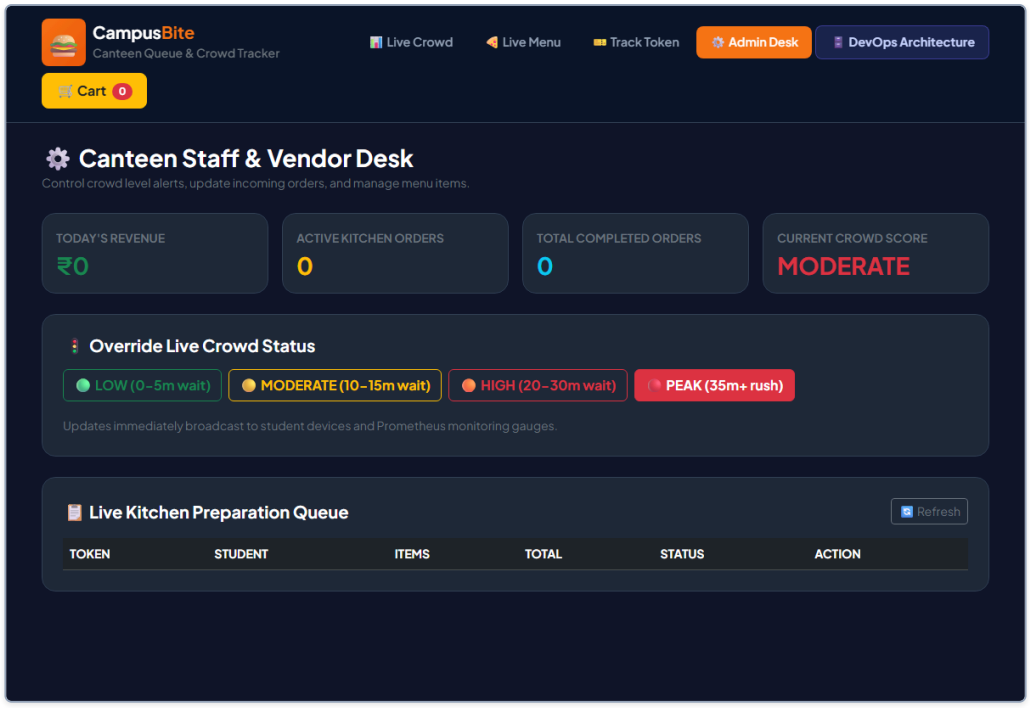


Fig. 9: Canteen vendor management console and live order dispatch

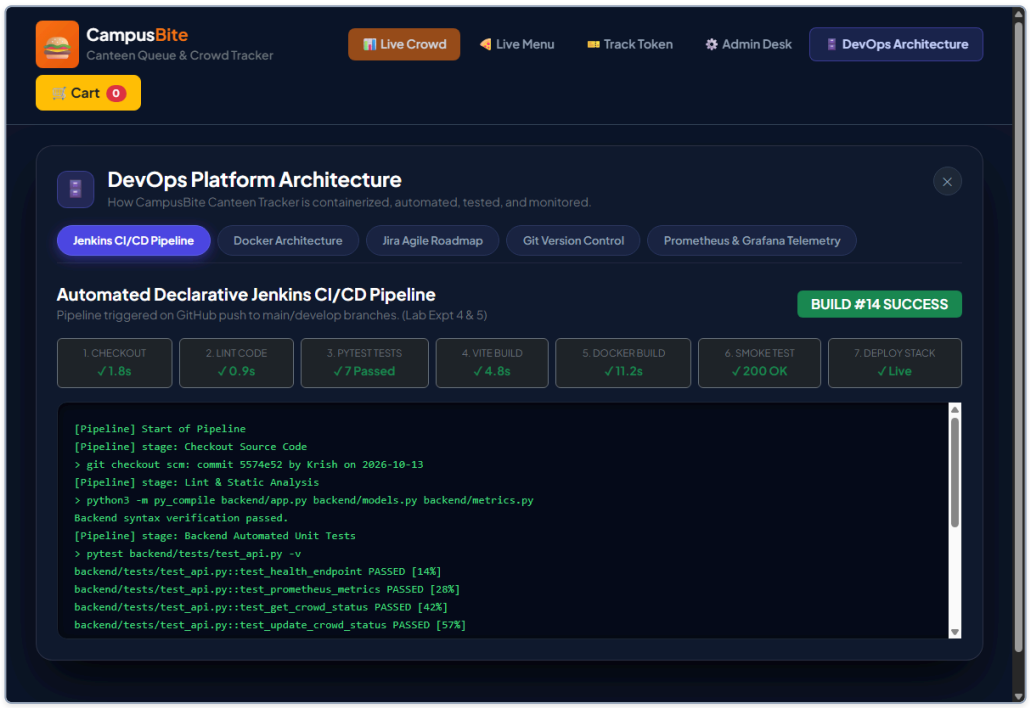


Fig. 10: Interactive DevOps Console: Jenkins CI/CD pipeline execution view

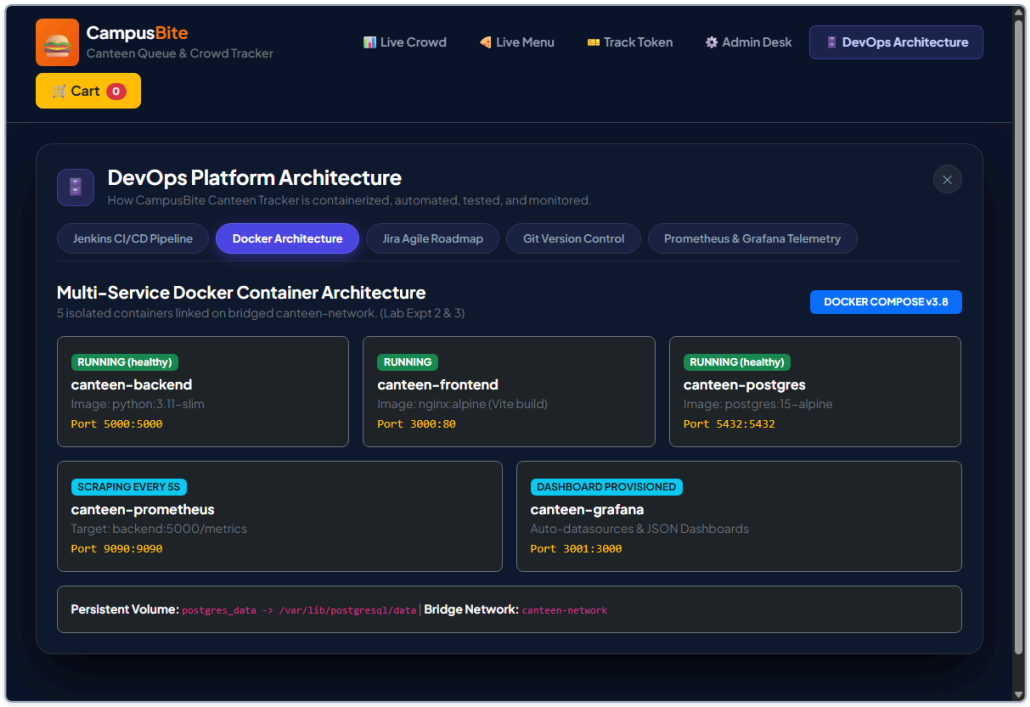


Fig. 11: Interactive DevOps Console: Docker multi-service architecture

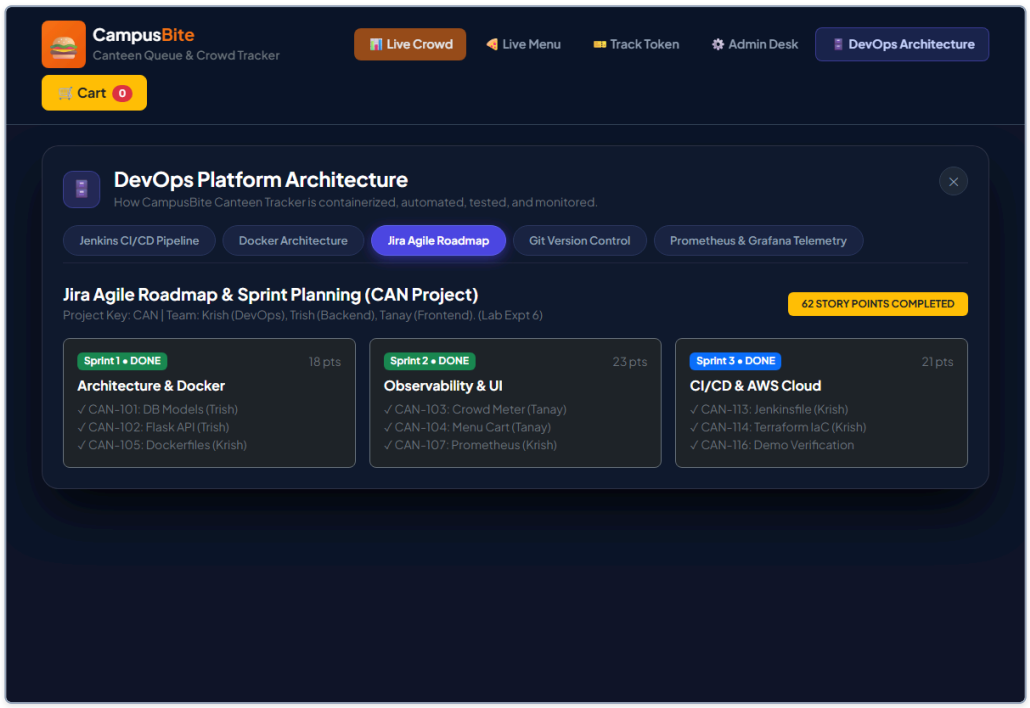


Fig. 12: Interactive DevOps Console: Jira Agile Scrum roadmap and sprint cards

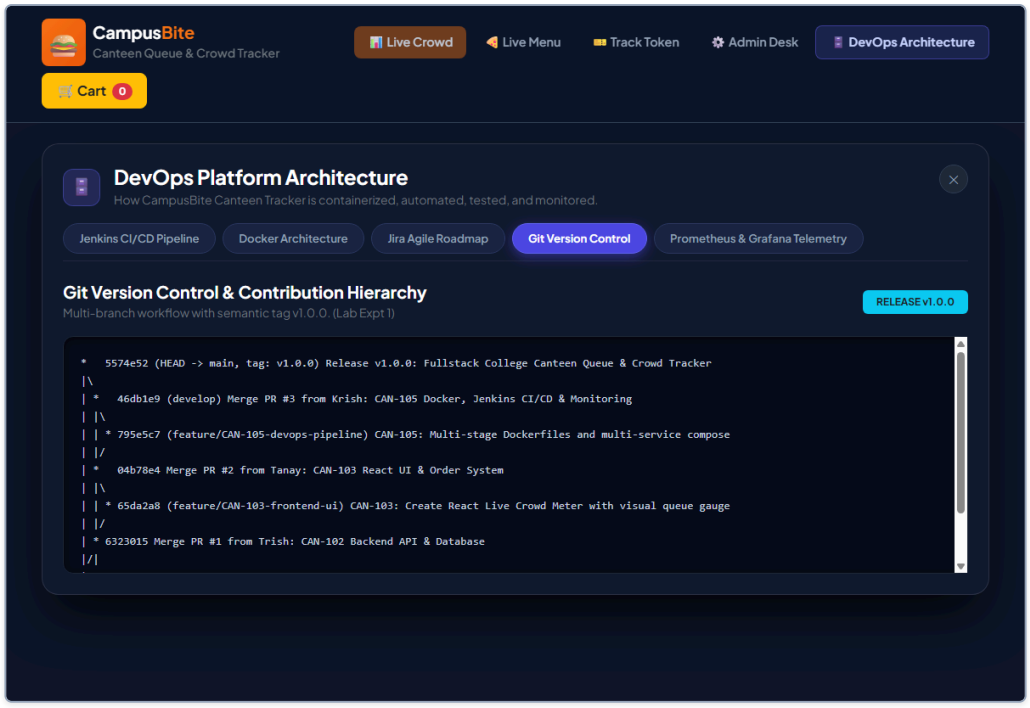


Fig. 13: Interactive DevOps Console: Git version control and rollback history

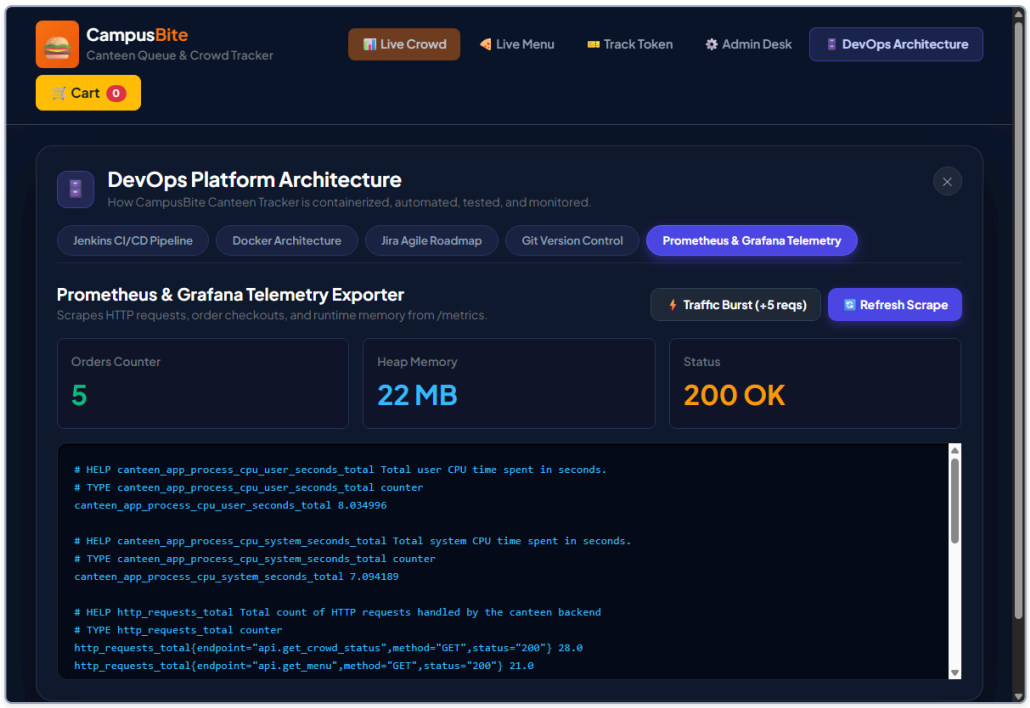


Fig. 14: Interactive DevOps Console: Prometheus & Grafana live telemetry with traffic burst

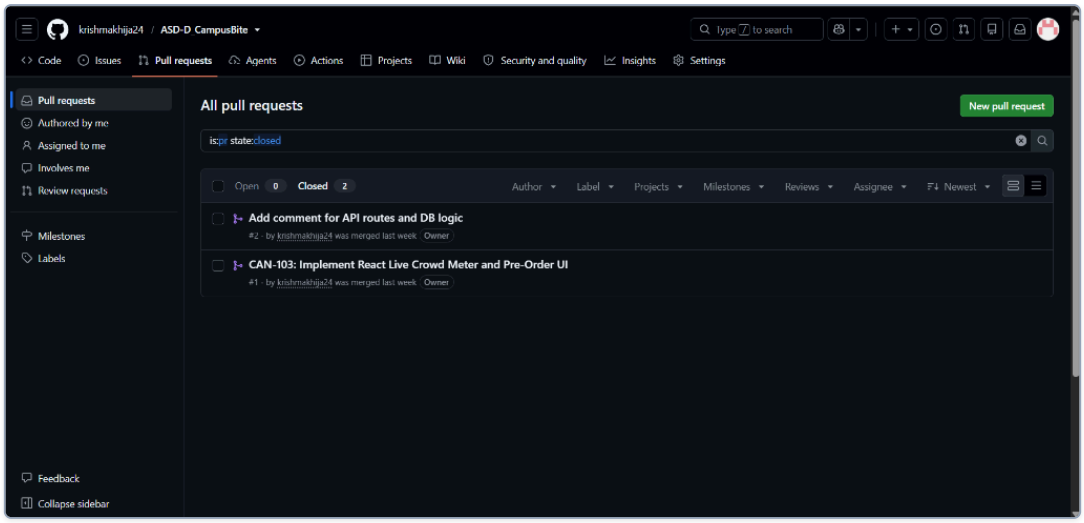


Fig. 15: GitHub Pull Requests screen showing team collaboration and merged PRs

3.5 Discussion and DevOps Integration

The CampusBite project demonstrates how individual laboratory experiments synthesize into a single, production-grade software engineering lifecycle. Rather than treating Git, Docker, Jenkins, Jira, and Prometheus as disconnected tools, CampusBite bridges development and operations: code committed by team members (**Tanay** on frontend, **Trish** on backend, **Krish** on DevOps) flows through Git branches, undergoes automated testing via Jenkins, gets containerized into Docker images, runs orchestrated via Docker Compose, and continuously streams telemetry to Grafana.

Area	Result Achieved
Version Control	Distributed Git repository with feature branches (`tanay`, `trish`), Pull Requests, semantic tag `v1.0.0`, and commit attribution across team members.
Containerization	Production-ready multi-stage Dockerfiles for frontend and backend with container healthcheck probes.
Multi-Service Deployment	Docker Compose coordinates 5 services (Frontend, Backend, PostgreSQL, Prometheus, Grafana) with automated startup dependencies and persistent volumes.
Continuous Integration	Automated Jenkins pipeline executing syntax linting and 7 unit tests (100% pass rate).
Automated Deployment	Extended pipeline automatically builds Docker images, executes ephemeral smoke tests, and deploys the application upon GitHub push.
Agile Workflow	Scrum backlog organized into 5 Epics and 16 User Stories across 3 Sprints with story point estimations.
Continuous Monitoring	Prometheus scrapes custom canteen metrics on `/metrics` every 5 seconds, visualized via Grafana dashboard panels.

Table 6: Summary of project results across laboratory areas

Chapter 4: Conclusion

4.1 Conclusion

The development of **CampusBite: A College Canteen Queue & Crowd Tracker** successfully met all functional and educational objectives. The platform delivers an effective, real-world digital solution to campus cafeteria congestion while demonstrating mastery over modern Agile and DevOps practices including Git, GitHub, Docker, Docker Compose, Jenkins, Jira, Prometheus, and Grafana.

4.2 Future Scope

1. **Automated WhatsApp/SMS Token Notifications:** Integrate Twilio or WhatsApp Business API to dispatch automated notifications when an order status advances to `READY_FOR_PICKUP`.
2. **Computer Vision Counter Queue Headcount:** Deploy overhead camera feeds with OpenCV/YOLO models to automatically update queue length metrics without vendor intervention.
3. **Kubernetes Cluster Autoscaling:** Transition Docker Compose service manifests to Kubernetes Deployments, Services, and Horizontal Pod Autoscalers (HPA) to scale pods during rush lunch hours.
4. **Cloud Infrastructure Provisioning:** Deploy the containerized application on AWS EC2 clusters using the authored Terraform configurations (`terraform/main.tf`).

4.3 Learning Outcomes

- End-to-end design, implementation, and deployment of a multi-tier web platform.
- Practical mastery of multi-branch Git team collaboration, code reviews, and rollbacks.
- Implementation of containerization, multi-service networking, and persistent storage volumes.
- Authoring declarative CI/CD pipelines with automated test gates and smoke validation.
- Application of Agile Scrum sprint planning, backlog grooming, and velocity tracking in Jira.
- Establishment of production observability via Prometheus scraping and Grafana dashboarding.

Chapter 5: References

[1] M. Fowler, “Continuous Integration,” *martinfowler.com*, 2006.

[2] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[3] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.

[4] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, 2014.

[5] K. Schwaber and J. Sutherland, “The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game,” *Scrum.org*, 2020.

[6] J. Turnbull, *The Monitoring and Observability Handbook*. Turnbull Press, 2020.

[7] Prometheus Authors, “Prometheus: An open-source monitoring system and time series database,” *prometheus.io*, 2024.

[8] Grafana Labs, “Grafana: The open observability platform,” *grafana.com*, 2024.

[9] Flask Documentation, “Flask: A lightweight WSGI web application framework,” *palletsprojects.com*, 2024.

[10] PostgreSQL Global Development Group, “PostgreSQL 15 Documentation,” *postgresql.org*, 2024.

[11] React Team, “React: The library for web and native user interfaces,” *react.dev*, 2024.

[12] Jenkins Project, “Jenkins User Documentation: Pipeline as Code,” *jenkins.io*, 2024.

[13] Atlassian, “Jira Software Agile Project Management Guide,” *atlassian.com/agile*, 2024.

[14] HashiCorp, “Terraform: Infrastructure as Code,” *terraform.io*, 2024.

[15] Pytest Development Team, “pytest: helps you write better programs,” *pytest.org*, 2024.

Appendix: Running the Project

A.1 Instant Local Run

```
cd backend
python -m pip install -r requirements.txt
python app.py
```

The application runs at `http://localhost:5000` with auto-seeded demo menu items, live orders, and Prometheus metrics.

A.2 Docker Multi-Service Compose Stack

```
docker compose config
docker compose up -d --build
docker compose ps
```

Frontend: `http://localhost:3000` | Backend API: `http://localhost:5000` | Prometheus: `http://localhost:9090` | Grafana: `http://localhost:3001` (admin/admin)

A.3 Automated Testing with Pytest

```
pytest backend/tests/test_api.py -v
```

A.4 Git Team Verification

```
git log --graph --oneline --all
git remote -v
```

A.5 Prometheus Metrics Verification

```
curl -s http://localhost:5000/metrics | grep canteen_
```